

Candidate Submission

AI USAGE LOG

AI Tooling Usage & Human Review Record

A record of where AI assistance was used during this assessment, what was generated versus independently decided, and the quality controls applied before anything was accepted into the codebase.

Prepared by

Youssef Bassiouny

Project

ProjectFlow — Full-Stack Engineer Assessment

Date

September 2026

CONTENTS

AI Log — ProjectFlow Assessment

Approach

Areas Where AI Was Used

Areas Where AI Was NOT Used

Quality Controls Applied

Summary

AI Log — ProjectFlow Assessment

This document records how AI tooling was used during this assessment, what was generated versus reviewed, and where human judgment was applied.

Approach

AI was used as a collaborative pair-programming assistant throughout the assessment. Every generated change was read, understood and verified before being accepted into the codebase. No AI output was committed without review.

The overall approach was:

1. Read and understand the codebase independently before making any changes.
 2. Use AI to draft implementations aligned with the existing patterns.
 3. Review every generated file carefully for correctness, security, and consistency.
 4. Run the full test suite after each change to confirm no regressions.
 5. Apply human judgment for all security-sensitive decisions.
-

Areas Where AI Was Used

Schema hardening (database layer)

AI generated the initial drafts for:

- `TaskActivity` schema (`task-activity.schema.ts`)
- `TaskSequence` schema (`task-sequence.schema.ts`)
- Adding `assignee` nullable reference to the `Task` schema
- Adding database-level compound indexes

Human review: All schema definitions were verified against the Mongoose documentation and the existing schema patterns in the repository. Index strategies were reviewed against the actual query patterns in the service layer.

Concurrency-safe task numbering

AI drafted the atomic `$inc` approach using `findOneAndUpdate` with `TaskSequence`.

Human review: The upsert/retry fallback logic was reviewed to ensure it correctly handles the case where the sequence document does not yet exist. The unique index on `TaskSequence.projectId` was confirmed to prevent duplicate sequence creation under concurrent writes.

Authorization fix — `PATCH /tasks/:taskId/status`

AI identified the authorization gap (missing `userId` extraction and missing `assertCanView` call in the original `updateStatus` handler) and drafted the fix.

Human review: The fix was verified by tracing the full request flow: controller → service → `ProjectAccessService.assertCanView` → `canView` evaluation. The regression test was reviewed to confirm it actually exercises the authorization path.

Assignee validation fix

AI identified a secondary bug: `PATCH /tasks/:taskId/assignee` used `projectMembersService.findRole` to validate the assignee, which only checks the `project_members` collection. Org OWNER/ADMIN users who have no explicit project membership row were incorrectly rejected.

Human review: The fix replaces `findRole` with `projectAccessService.resolve` + `canView`, which is the canonical access-check used everywhere else in the system. This correctly allows org OWNER/ADMIN to be assigned while still rejecting users with no access at all.

Task assignment API

AI drafted:

- `UpdateTaskAssigneeDto` DTO
- `ListActivityQueryDto` DTO
- `TasksService.assignTask` method
- `TasksService.findActivity` method with cursor-based pagination
- `TasksController` handler additions

Human review: Authorization logic was scrutinized carefully:

- Actor check: `assertCanView` verifies project access.
 - Role check: `canManage` guards assigning another user (MEMBER can only self-assign).
 - Assignee check: full `canView` check confirms assignee has project access (not just a `project_members` row).
 - Activity creation: reviewed for consistency — no activity is recorded when the assignment does not change.
-

E2E test suite

AI drafted the test cases in `tasks.e2e.spec.ts`.

Human review: Every test scenario was verified against the backend authorization rules:

- Self-assignment by MEMBER
 - Assignment by PROJECT_MANAGER
 - Assignment by org OWNER (no explicit project row)
 - Rejection of MEMBER assigning another user
 - Rejection of outsider accessing the assign endpoint
 - Rejection of assigning a non-project user
 - Activity creation on assignment
 - Activity isolation between tasks
 - Cursor-based pagination of activity
-

Frontend components

AI drafted:

- `AssigneeSelect` component (`assignee-select.tsx`)
- `ActivityTimeline` component (`activity-timeline.tsx`)
- `useTaskActivity` hook (React Query v5 `useInfiniteQuery`)
- `useUpdateTaskAssignee` mutation hook
- `fetchTaskActivity` API function

Human review: The React Query v5 API changes (object-form `useInfiniteQuery` with `initialPageParam`) were verified against the installed version (5.102.8). TypeScript errors from the original v4-style API were identified and corrected. The `ProjectRole` comparison in `AssigneeSelect` was corrected from string literals to the enum values.

Production preparation

AI drafted:

- `railway.json` — minimal Railway deployment configuration
- `vercel.json` — monorepo-aware Vercel configuration
- `PORT` / `API_PORT` fallback in `main.ts` for Railway compatibility
- Updated `.env.example` with production placeholders
- Updated `README.md` with deployment section

Human review: The Railway PORT behavior was verified against Railway's documentation (Railway injects `PORT`, not a custom variable). The Vercel monorepo build command sequence (shared package first, then web app) was verified against the actual pnpm workspace structure.

Areas Where AI Was NOT Used

- **Security decisions:** All authorization boundary decisions were made by the engineer. AI was used to implement the decision, not to make it.
- **Architecture decisions:** The existing architecture was intentionally preserved. Decisions to stay with MongoDB + NestJS + Mongoose (rather than adding Redis, Kafka, etc.) were made independently.
- **Assessment compliance judgment:** Whether a feature is required or optional was judged independently from the assessment specification.

Quality Controls Applied

Control	Method
Type safety	<code>pnpm typecheck</code> — zero errors required
Lint	<code>pnpm lint</code> — zero errors required
Tests	<code>pnpm test</code> — all 49 tests pass
Build	<code>pnpm build</code> — production build succeeds
Manual review	Every generated file read and understood before acceptance

Summary

AI was used to increase implementation speed while maintaining correctness. Every meaningful change — especially in authorization logic — was reviewed manually before being accepted. The resulting code is consistent with the existing codebase patterns and passes the full automated test suite.